Summary

Browser Resource Loading Priority Differences

Resource loading behaves so differently across browsers because of undocumented, non-standard two-phase loading ("pipeline mode"/"tight mode") logic that each engine implements its own way. The core message: the same HTML produces radically, sometimes opposite, loading waterfalls across Chrome, Safari and Firefox, making predictable cross-browser performance nearly impossible. `fetchpriority` is the main mitigation tool, but it too behaves inconsistently per engine.

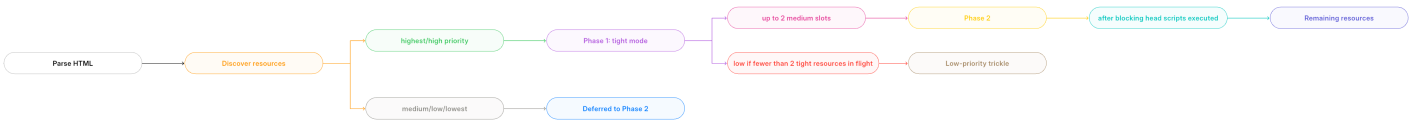
How browsers should load, and priority signals

- "Render-blocking" CSS and JavaScript must load in full before the page renders (page stays blank until done); deferred JavaScript executes after the page loads, and `preload` hints tell the browser about later-discovered files like fonts or CSS.
- The naive load walks the page top to bottom and sends an HTTP request per resource; HTTP two and three allow requesting in parallel over a single connection, but the server can't respond to all in parallel — bandwidth, and congestion control at connection start, still constrain it.
- The naive order makes render-blocking JavaScript at the bottom the last thing loaded, so the page feels slow; the fix is to tell the server which requests matter, which the protocol calls "priority".
- Priority roughly: render-blocking CSS = highest; render-blocking JavaScript = high; deferred JavaScript = low. In practice, browsers send these signals well, but servers often ignore them.
- The speaker's research began about five years ago; in testing popular servers, Apache was almost correct, NGINX "not really," and Node.js completely ignored the browser's signals and did "one of the worst possible things" for web performance.
- A follow-up study across more servers and hosting providers found only two did it correctly, though the rest weren't all Node.js-level wrong — there's a scale.
- Observed in the wild: three images requested top to bottom came back in reverse order (3, 2, 1), delaying the largest Contentful Paint image; and in a more complex case, render-blocking JavaScript and important fonts were delayed while the server gave bandwidth to images.

Two-phase loading ("pipeline"/"tight mode") — undocumented, non-standard

- You can't stop bad servers; you can only mitigate. Browsers take roughly the top ten to twenty most important high-priority resources first and only request everything else once those are done, so a bad server can only get it wrong within that block.
- None of this is a web standard, written down, or agreed by web developers — each vendor did its own thing. For a long time it wasn't documented at all; the only source is a Google Doc by the Chromium engineer who implemented it.
- The talk is roughly ten percent of the speaker's research findings.

Chrome two-phase loading (tight mode)



Chrome

- The Google Doc says Chrome loads in two phases; tight mode is the initial phase constraining low-priority resources until "the body is attached to the document," which the author glosses as "after all blocking scripts in the head have been executed".
- Another sentence — low-priority resources only load if fewer than two tight resources are in flight at discovery — the speaker initially skipped and returned to later.
- Chrome's type priorities: HTML and CSS highest; render-blocking JavaScript high; deferred JavaScript low; default images low. "Lower priority" in the doc means anything not high-to-highest — medium, low or lowest is delayed out of the first phase.
- Test page: two render-blocking scripts with an artificial ~2.5 s server delay, plus ten images. Chrome requested the scripts first and only requested the images once they finished — the browser actively delays the requests.
- Change (~2–3.5 years ago to today): Chrome now gives the first five images medium priority (they were low) to improve LCP, since the LCP image is usually among the first five images.
- To accommodate that, Chrome also loads two medium-priority resources during tight mode: the first two of the five images load, two slots open for the next two, and the last of the five waits.
- With the first script delayed only 1 s instead of 2.5 s, only one tight request remains in flight, triggering the "fewer than two" rule: low-priority loads start, but only one at a time, producing a visible stepping waterfall. This was added so a single large, slow resource can't hold back the whole page.

- Surprise: only JavaScript triggers pipeline mode — swapping the head scripts for CSS means everything is requested at once, even though CSS is equally render-blocking and the page stays blank. This surprised even the Google Chrome developer maintaining the code, who thought CSS also triggers pipeline mode.
- On the simple page (two render-blocking scripts, two deferred scripts, ten images): tight mode loads only the high-priority render-blocking scripts plus the five medium images; deferred JavaScript waits for the second phase.
- The speaker notes this inverts the old intuition that the head is more important than the body and JavaScript more important than anything — a junior developer 20 years ago would be very confused.

Safari

- Five years on, still zero documentation; everything here is the speaker's experimentation.
- Safari does do tight mode and delays images, but lacks Chrome's first-five-images change — presumably because LCP is a Google metric — and it does apply the "load low-priority when only one thing in flight" rule.
- Safari does trigger tight mode for CSS, the opposite of Chrome, delaying images until most CSS is done; the speaker thinks Safari is right and this is the more logical behavior.
- Four words matter: "script in the head." Moving scripts to the top of the body behaves the same visually (blank page) but Chrome sees no JavaScript in the head and drops tight mode entirely, loading everything at once; Safari still triggers and delays.
- Scripts at the bottom: both browsers delay, "finally makes sense".
- Scripts in the middle of the body: the speaker expected everything above to load together and images after the script to be delayed; instead Safari applies some odd heuristic — images get requested, then the rest delayed even before the script is reached. He tried many pages, couldn't work out the logic, and gave up after a long time.
- On the simple page, Safari requests deferred JavaScript alongside synchronous JavaScript in tight mode and delays images until the last possible moment; the speaker dislikes this since deferred JS is only needed after the page downloads, and bandwidth is wasted on it.
- Deferred JavaScript alone doesn't trigger tight mode, and images then load alongside it; the odd behavior only appears combined with synchronous, important JavaScript.

Firefox

- Firefox does not do tight mode at all: everything in the HTML is requested at the same time, no delays.
- This does not make Firefox slower or worse. The speaker's earlier YouTube talk used that graphic as the thumbnail, went semi-viral, and drew heavy criticism from Firefox fans.
- Setting the record straight: one can argue Firefox is the purest browser, using the standards as intended; with a compliant server, Firefox can be a lot faster (e.g., a server knowing everything it might need and pre-fetching from origin, CDN setups).

- The point is not better or worse — it's that they're different, in very different ways.

Mitigation: `fetchpriority`

- With the same HTML loading differently per engine, it's almost impossible to craft a page with predictable performance across Chrome, Safari and Firefox.
- `fetchpriority` (added a couple of years ago) can be set to `high` or `low` on various HTML tags — a relative hint to raise or lower the priority the browser would normally assign, not an absolute setting like highest or lowest.
- `fetchpriority="high"` moves a resource into the high phase in Chrome, which is very strict; in Safari it's less flexible but mostly works the same.
- Example: a hero/LCP image normally loads in Safari's second phase; with `fetchpriority="high"` Safari moves it into the high phase and requests it as soon as possible. Chrome also does this — the speaker says he should have used five images rather than one to make it clearer, though it still moves from medium to high.
- Lowering works in Chrome: `fetchpriority="low"` moves a resource out of the high phase. The speaker could find nothing in Safari to achieve the same, e.g. to push deferred JavaScript into the second phase.
- In Safari, `fetchpriority="low"` on the deferred script lowers priority from high to medium (visible in the trace) but keeps it loading in the first phase.
- Firefox, which requests everything at once, benefits most from fine-grained priority control and has the most advanced/flexible system, with a good documentation page.
- You can't optimize only for Firefox and expect other browsers to behave the same — nothing is consistent on the web.
- Browsers don't even agree on initial priorities: a normal font is highest priority in Chrome but low in Firefox.
- Preloading a font: Chrome treats it as a signal to lower priority (high is already high); Firefox treats it as a signal to raise from low to medium; Safari ignores developers.
- Adding `fetchpriority="high"` to a preloaded font: Chrome ignores it and stays at high, while Firefox and Safari bump up to high.
- `fetchpriority="low"` on the preload: Chrome listens and drops it two levels; Firefox and Safari both drop it one level.
- JavaScript is another example: in Safari almost all JavaScript is high priority, including defer and async, while Chrome and Firefox do very different things with it.
- Also researched but not covered today: behavior across multiple connections and more. The speaker has an in-depth blog post on these details.

Key takeaways and framing

- Using the exact same HTML yields radically different — sometimes opposite — behavior across engines, making consistent loading nearly impossible for web developers.

- The speaker's headline statements are not deranged: following recent web standards and best practices like CDNs should give good performance across browsers and devices, which isn't what the current web delivers.
- Perspective: this has always been the case — Internet Explorer 6 and table layouts, and a period when some sites only worked in Chrome.
- Today it's better: pages at least load; most of the audience knew none of this, which is good, because it can be abstracted away.
- The conference theme is "legacy" — the web is nothing but legacy, standing on what was built before.
- Chrome and Safari share the basics of the pipeline because Chrome forked WebKit (which became Blink); the mechanisms come from the original WebKit and still echo today.
- In HTTP/1 every resource had its own connection, so which resource you requested first mattered far more than in HTTP 2/3, where late render-blocking JavaScript could cost you very badly.

Open questions

- Whether the "script in the head" wording is the real trigger, and what logic Safari applies with scripts mid-body, remains unresolved — the speaker gave up after long experimentation.
- Why CSS doesn't trigger Chrome's tight mode while it does trigger Safari's remains unexplained, and confused even the Chrome maintainer.
- How to move Safari's deferred JavaScript out of the first phase via `fetchpriority` — not found.

Next steps

- For deeper detail, read the speaker's blog post on font and JavaScript priority behavior. No owner or deadline was stated.
- No tasks or commitments were assigned to anyone in this recording.